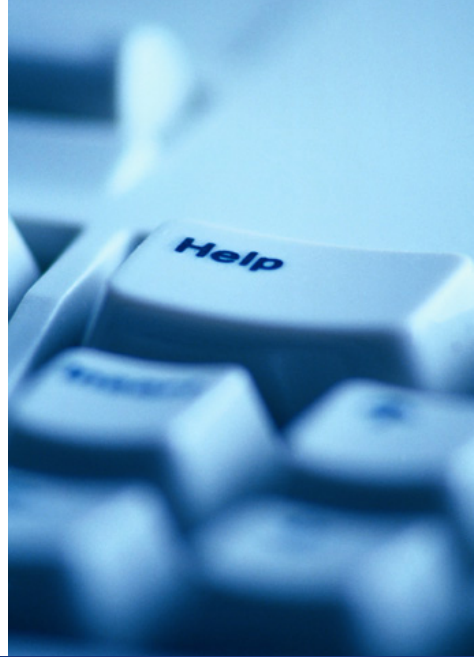




操作系统原理

Operating Systems Principles

陈鹏飞
计算机学院



中山大學

计算机学院 (软件学院)

SUN YAT-SEN UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



第十二讲 — I/O管理





目标

- 剖析操作系统的I/O子系统;
- 讨论I/O硬件的原理和复杂性;
- 解释I/O硬件和软件的性能问题;

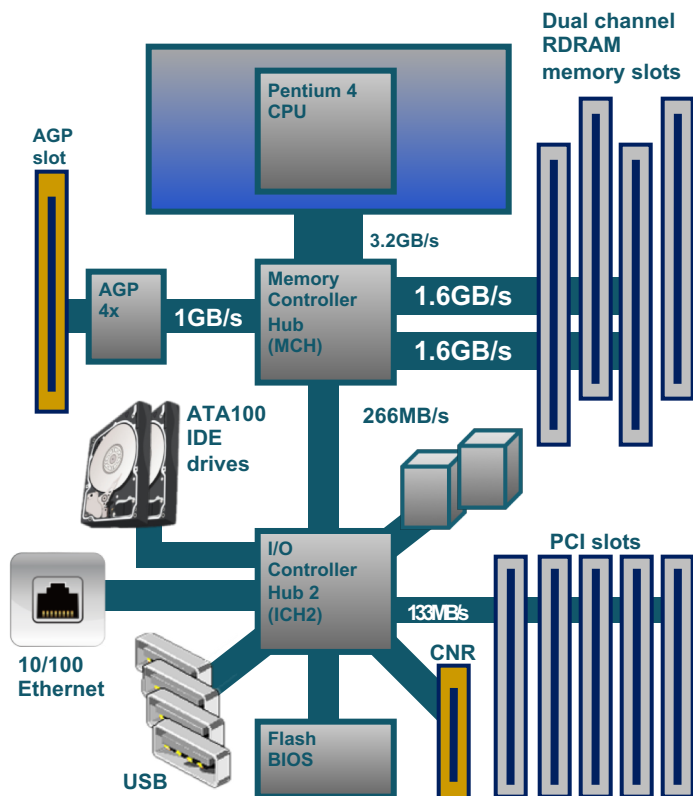


概述

- ❖ I/O管理是操作系统设计和运行的主要组成部分
 - 计算机运行的重要方面;
 - I/O设备差异很大;
 - 控制它们的各种方法, 构成了I/O子系统;
 - 性能管理;
 - 新型设备频繁出现;
- ❖ I/O设备的要素: 端口、总线、设备控制器连接到各种设备
- ❖ 设备驱动程序封装了设备详细信息;
 - 向I/O子系统提供统一的设备访问接口



I/O结构



北桥

- ▶ 内存/CPU
- ▶ AGP/PCI-Express
- ▶ Built-in display

南桥

- ▶ ATA/IDE
- ▶ PCI总线
- ▶ USB/Firewire总线
- ▶ Serial/Parallel接口
- ▶ DMA 控制器
- ▶ Interrupt控制器
- ▶ RTC, ACPI, BIOS, ...



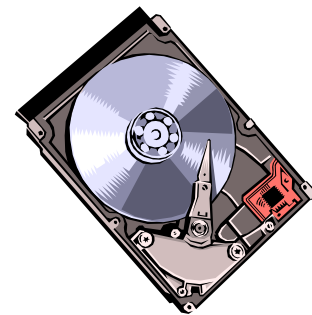
I/O设备分类

► 计算机系统中参与I/O的外设大体上分为3类:

- 存储
- 传输
- 人机交互

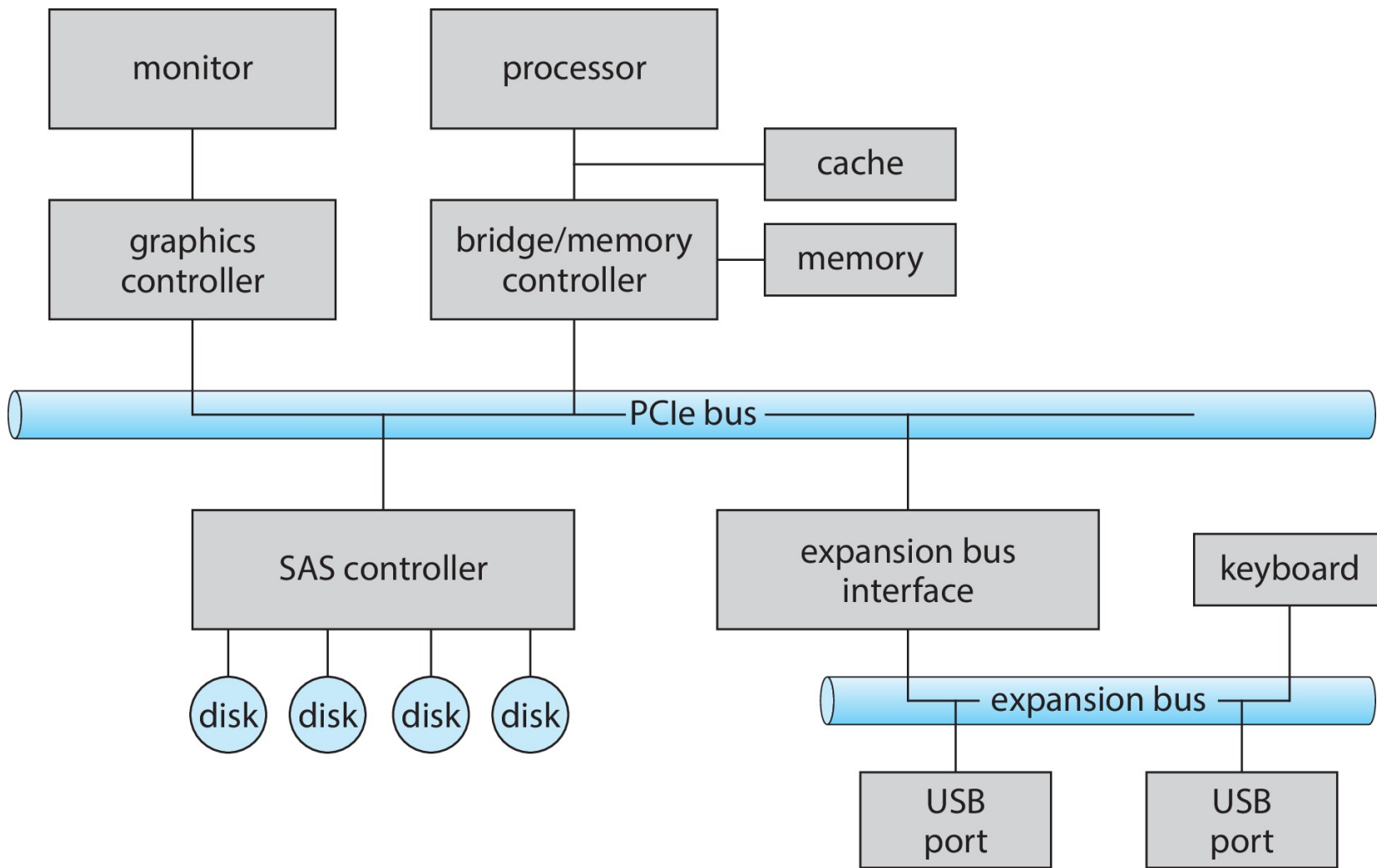
❖ 通用概念 - I/O设备与计算机接口的信号

- 端口 - 设备的连接点(地址)
- 总线 - 一组线路和通过线路传输信息的严格定义的协议;
 - PC和服务中常见的PCI总线, PCI Express (PCIe)
 - 扩展总线连接相对较慢的设备;
 - 串行连接SCSI (SAS) 公共磁盘接口;





I/O 总线





I/O设备之间的差别

- ▶ 各类别设备之间的差别很大，同一类别不同设备差别也很大

Data Rate

- there may be differences of magnitude between the data transfer rates

Application

- the use to which a device is put has an influence on the software

Complexity of Control

- the effect on the operating system is filtered by the complexity of the I/O module that controls the device

Unit of Transfer

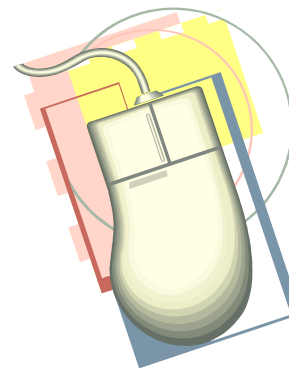
- data may be transferred as a stream of bytes or characters or in larger blocks

Data Representation

- different data encoding schemes are used by different devices

Error Conditions

- the nature of errors, the way in which they are reported, their consequences, and available range of responses differs from one device to another



the



典型I/O设备的速率

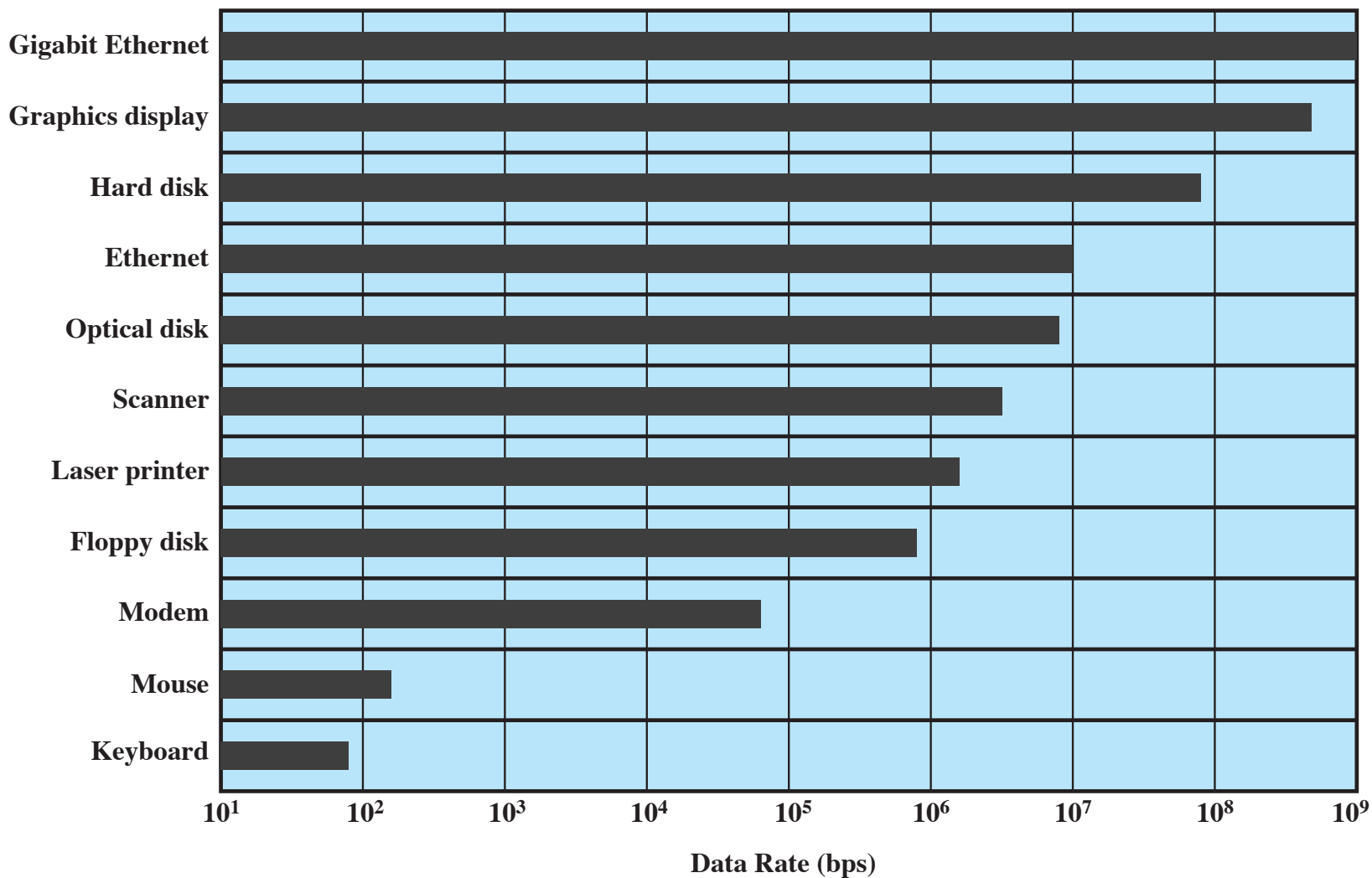


Figure 11.1 Typical I/O Device Data Rates



I/O 控制器

- 控制器（主机适配器）-操作端口、总线、设备的电子设备
 - 有时是集成的；
 - 有时单独的电路板（主机适配器）；
 - 包含处理器、微码、专用存储器、总线控制器等；
- 有些与带有总线控制器、微码、内存等的单个设备控制器进行通信。



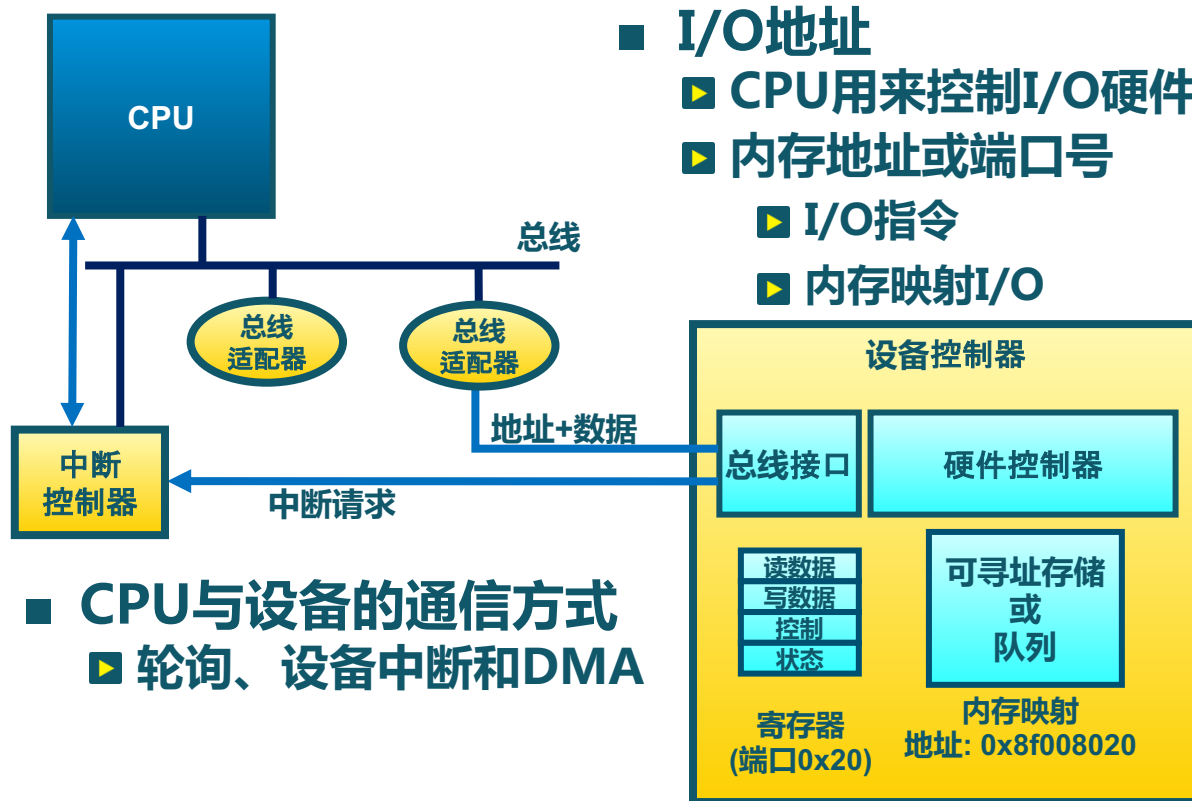
I/O硬件

- ❖ 光纤通道 (FC) 是一种复杂的控制器，通常是单独的电路板 (主机总线适配器、HBA) 插入总线；
- ❖ I/O指令控制设备；
- ❖ 设备通常有寄存器，设备驱动程序在其中放置命令、地址和数据，以便在命令执行后从寄存器中写入或读取数据；
- ❖ 寄存器类型：数据输入寄存器、数据输出寄存器、状态寄存器、控制寄存器；
- ❖ 通常为1-4字节，或FIFO缓冲区；



I/O结构

- 设备控制器
 - ▶ CPU和I/O设备间的接口
 - ▶ 向CPU提供特殊指令和寄存器
- I/O地址
 - ▶ CPU用来控制I/O硬件
 - ▶ 内存地址或端口号
 - ▶ I/O指令
 - ▶ 内存映射I/O



- CPU与设备的通信方式
 - ▶ 轮询、设备中断和DMA



I/O指令和内存映射I/O

■ I/O指令

- ▶ 通过I/O端口号访问设备寄存器
- ▶ 特殊的CPU指令
 - ▶ `out 0x21, AL`

■ 内存映射I/O

- ▶ 设备的寄存器/存储被映射到内存物理地址空间中;
- ▶ 通过内存load/store指令完成I/O操作;
- ▶ MMU设置映射, 硬件跳线或程序在启动时设置地址;

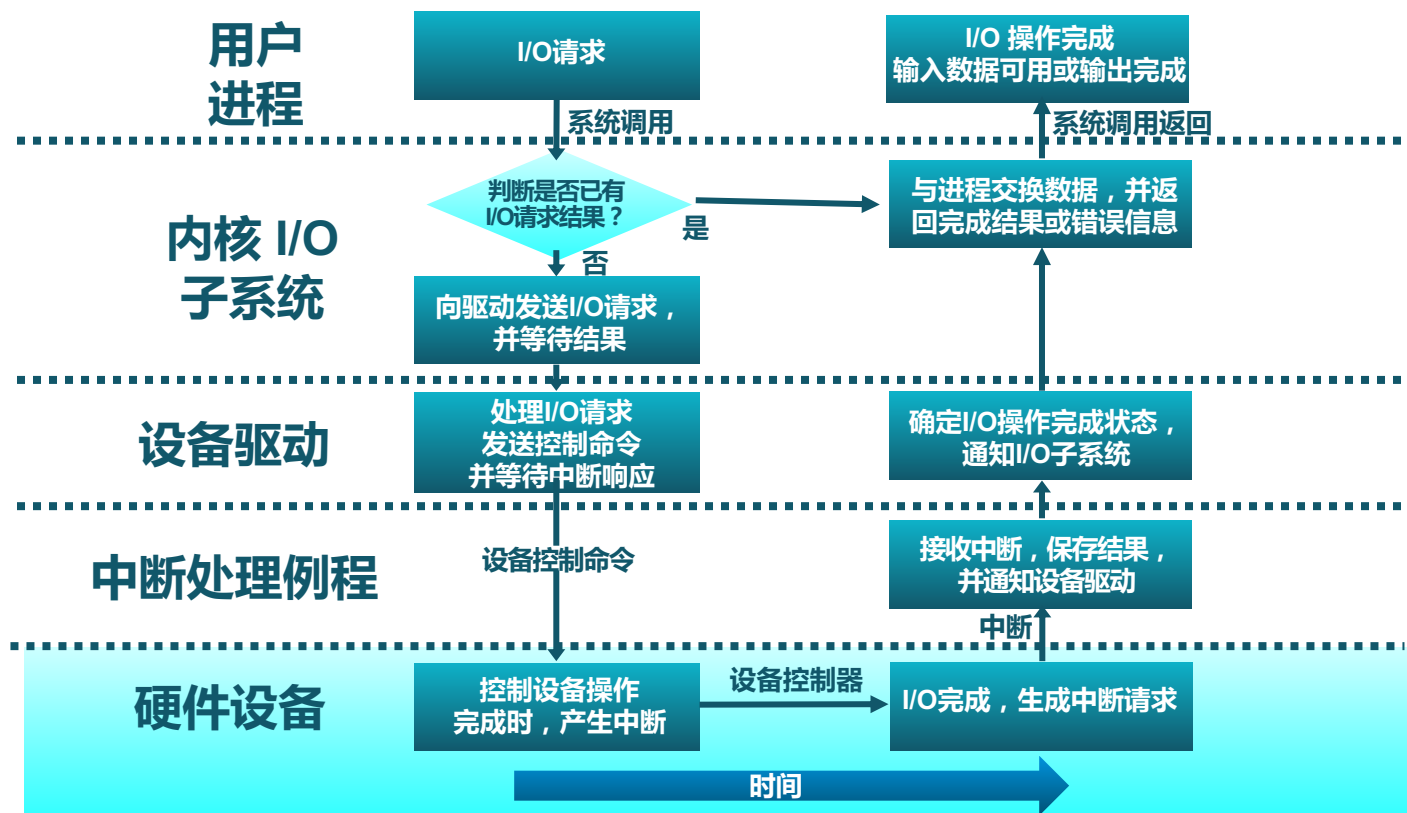


PC设备中的I/O端口位置

I/O address range (hexadecimal)	device
000–00F	DMA controller
020–021	interrupt controller
040–043	timer
200–20F	game controller
2F8–2FF	serial port (secondary)
320–32F	hard-disk controller
378–37F	parallel port
3D0–3DF	graphics controller
3F0–3F7	diskette-drive controller
3F8–3FF	serial port (primary)



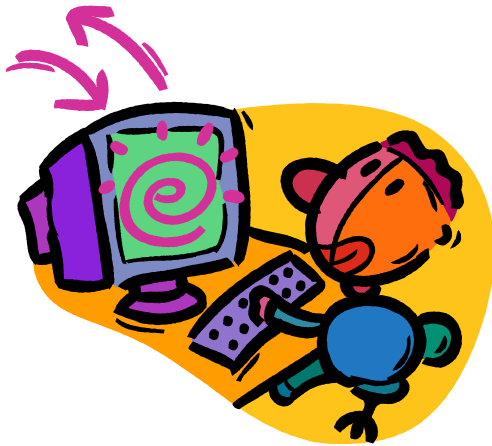
I/O请求生存周期





I/O 技术

Table 11.1 I/O Techniques



	No Interrupts	Use of Interrupts
I/O-to-memory transfer through processor	Programmed I/O 轮询 (polling)	Interrupt-driven I/O
Direct I/O-to-memory transfer		Direct memory access (DMA)



轮询

❖ 对于I/O的每个字节

1. 从状态寄存器读取忙位，直到该位清零；
2. 主机设置读或写位，如果写入，则将数据复制到数据输出寄存器中；
3. 主机设置命令就绪位；
4. 控制器设置忙位；
5. 控制器读取命令寄存器，并看到命令。从数据输出寄存器中读取一个字节，并向设备执行I/O操作；
6. 传输完成时，控制器清除忙位、错误位、命令准备位；

❖ 步骤1是等待设备I/O的忙等待周期

- 如果设备速度快，则合理
- 但如果设备运行缓慢，则效率低下，CPU资源浪费；
- CPU切换到其他任务？
 - 但如果错过一个周期，数据将被覆盖/丢失



中断I/O

❖ 轮询可以在3个指令周期内发生

- 读取状态、逻辑和以提取状态位，如果不是零，则分支
- 如果设备很少被发现就绪，如何提高效率？

❖ 由I/O设备触发的CPU中断请求方法

- 处理器在每条指令后进行检查

❖ 中断处理程序接收中断

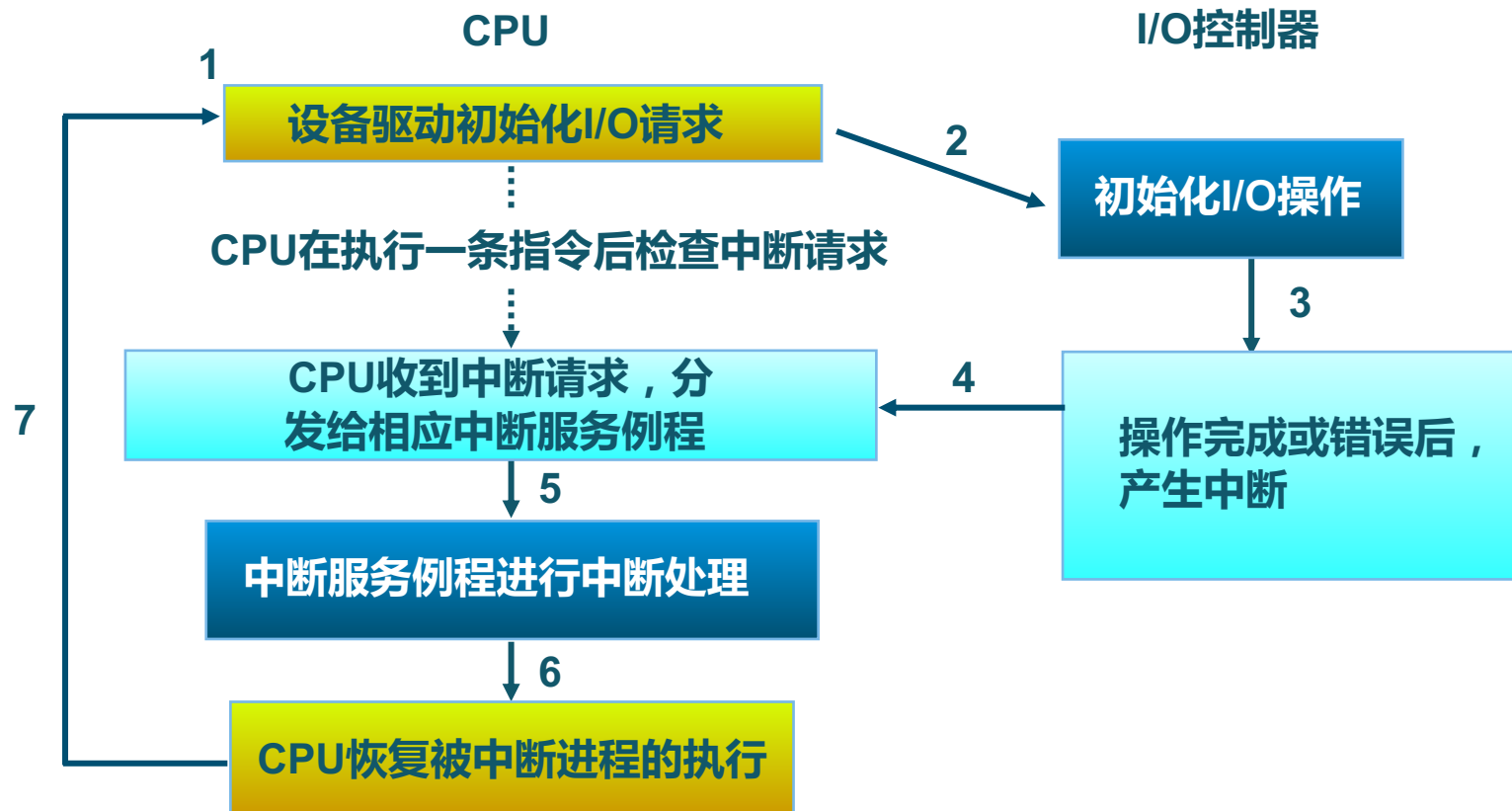
- 可以忽略或延迟某些中断

❖ 中断向量将中断分派给正确的处理程序

- 开始和结束时的上下文切换
- 基于优先级
- 两条中断请求线：非屏蔽中断和可屏蔽中断；
- 如果多个设备处于同一中断编号，则形成中断链；



中断I/O过程





中断的其他用处

- ❖ **Interrupt mechanism also used for exceptions**
 - Terminate process, crash system due to hardware error
- ❖ **Page fault executes when memory access error**
- ❖ **System call executes via trap to trigger kernel to execute request**
- ❖ **Multi-CPU systems can process interrupts concurrently**
 - If operating system designed to handle it
- ❖ **Used for time-sensitive processing, frequent, must be fast**



中断向量

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

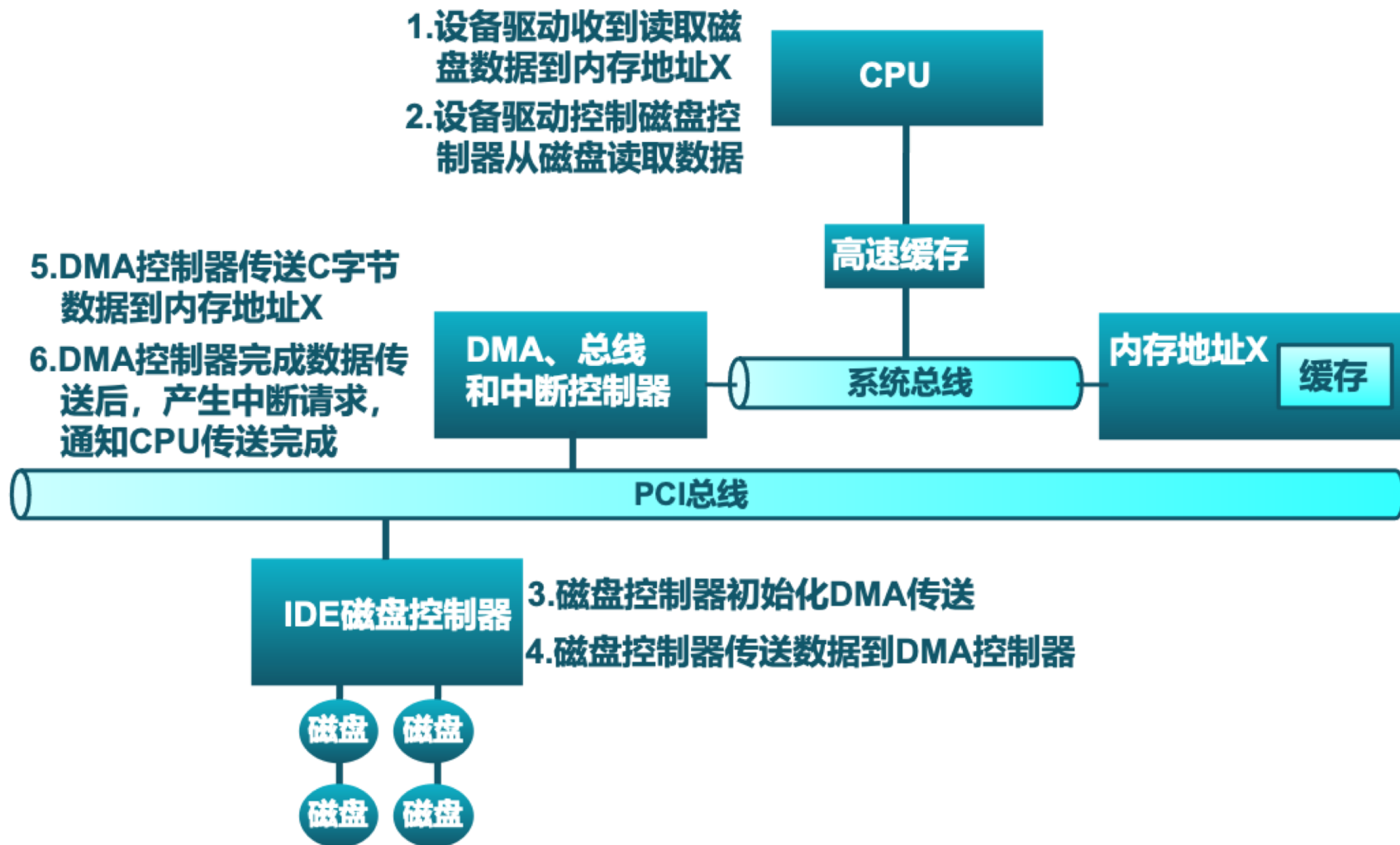


DMA

- ❖ Used to avoid programmed I/O (one byte at a time) for large data movement
- ❖ Requires **DMA controller**
- ❖ **Bypasses CPU** to transfer data directly between I/O device and memory
- ❖ OS writes **DMA command block** into memory
 - Source and destination addresses
 - Read or write mode
 - Count of bytes
 - Writes location of command block to DMA controller
 - Bus mastering of DMA controller – grabs bus from CPU
 - **Cycle stealing** from CPU but still much more efficient
 - When done, interrupts to signal completion
- ❖ Version that is aware of virtual addresses can be even more efficient -



DMA过程举例



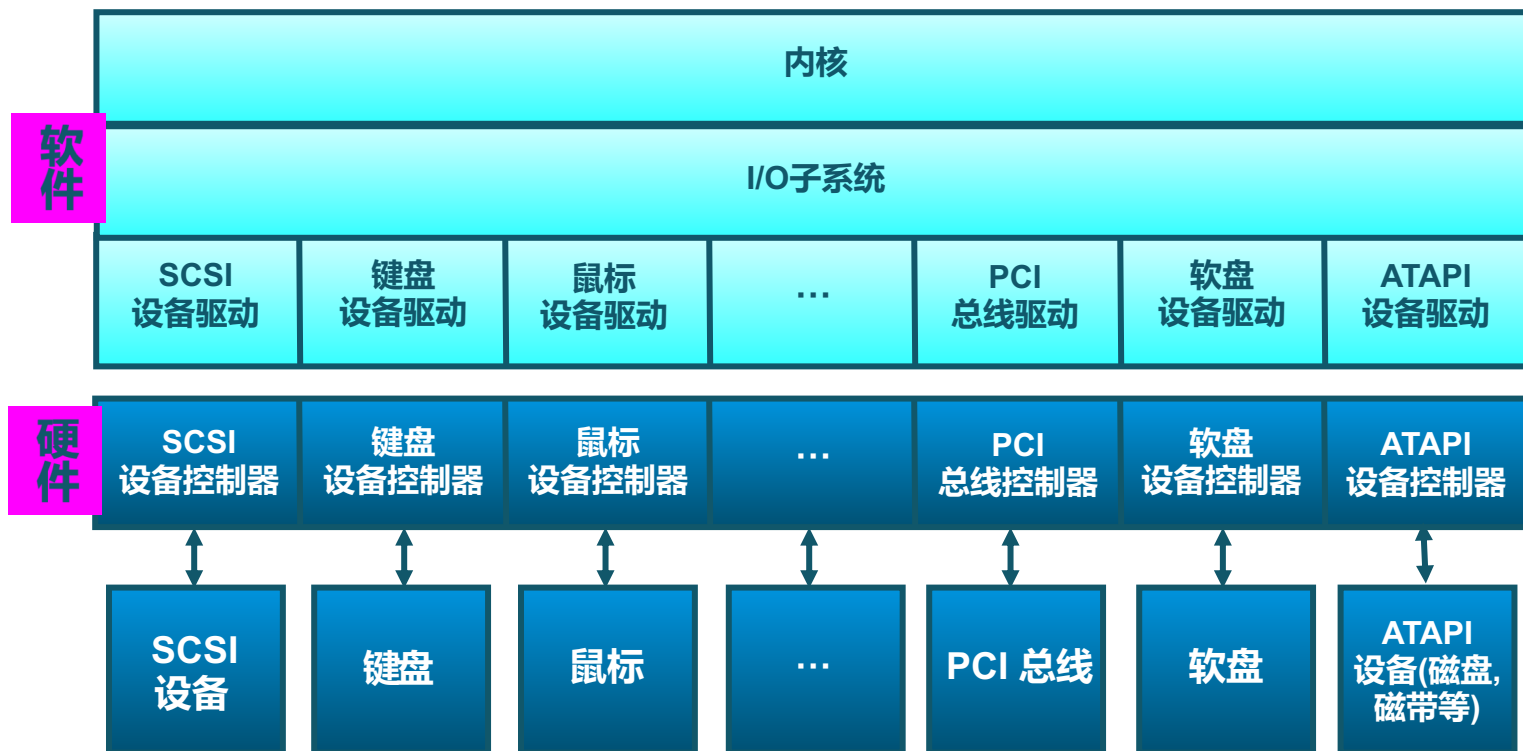


应用程序I/O接口

- ❖ I/O系统调用将设备行为封装在泛型类中;
- ❖ 设备驱动程序层隐藏了内核中I/O控制器之间的差异;
- ❖ 使用现成协议的新设备不需要额外的工作;
- ❖ 每个操作系统都有自己的I/O子系统结构和设备驱动程序框架
- ❖ 设备在许多方面都有所不同
 - 字符流或块
 - 顺序或随机访问
 - 同步或异步 (或两者兼有)
 - 共享或专用
 - 操作速度
 - 读写、只读或只写



内核I/O结构





I/O设备的特点

aspect	variation	example
data-transfer mode	character block	terminal disk
access method	sequential random	modem CD-ROM
transfer schedule	synchronous asynchronous	tape keyboard
sharing	dedicated sharable	tape keyboard
device speed	latency seek time transfer rate delay between operations	
I/O direction	read only write only read-write	CD-ROM graphics controller disk



I/O设备的特点

- ❖ 设备驱动程序处理的设备的微妙之处
- ❖ 大体上, I/O设备可以被操作系统分组为
 - 块I/O
 - 字符I/O (流)
 - 内存映射文件访问;
 - 网络套接字;
- ❖ 对于I/O设备特定特性的直接操作, 通常为逃生/后门
 - Unix `ioctl()` 调用将任意位发送到设备控制寄存器, 并将数据发送到设备数据寄存器
- ❖ UNIX和Linux使用“主要”和“次要”设备编号的元组来标识设备的类型和实例 (此处主要8和次要0-4) `%ls -l /dev/sda*`

```
brw-rw---- 1 root disk 8, 0 Mar 16 09:18 /dev/sda
brw-rw---- 1 root disk 8, 1 Mar 16 09:18 /dev/sda1
brw-rw---- 1 root disk 8, 2 Mar 16 09:18 /dev/sda2
brw-rw---- 1 root disk 8, 3 Mar 16 09:18 /dev/sda3
```




块与字符设备

❖ Block devices include disk drives

- Commands include read, write, seek
- **Raw I/O, direct I/O**, or file-system access
- Memory-mapped file access possible
 - File mapped to virtual memory and clusters brought via demand paging
- DMA

❖ Character devices include keyboards, mice, serial ports

- Commands include **get()**, **put()**
- Libraries layered on top allow line editing



网络设备

- ❖ **Varying enough from block and character to have own interface connect, accept, send, receive, ...**
- ❖ **Linux, Unix, Windows and many others include socket interface**
 - Separates network protocol from network operation
 - Includes **select()** functionality, 支持服务器应用, 管理一组套接字;
- ❖ **Approaches vary widely (pipes, FIFOs, streams, queues, mailboxes)**



时钟和定时器

- ❖ Provide current time, elapsed time, timer
- ❖ Normal resolution about 1/60 second
- ❖ Some systems provide higher-resolution timers
- ❖ Programmable interval timer used for timings, periodic interrupts
- ❖ `ioctl()` (on UNIX) covers odd aspects of I/O such as clocks and timers

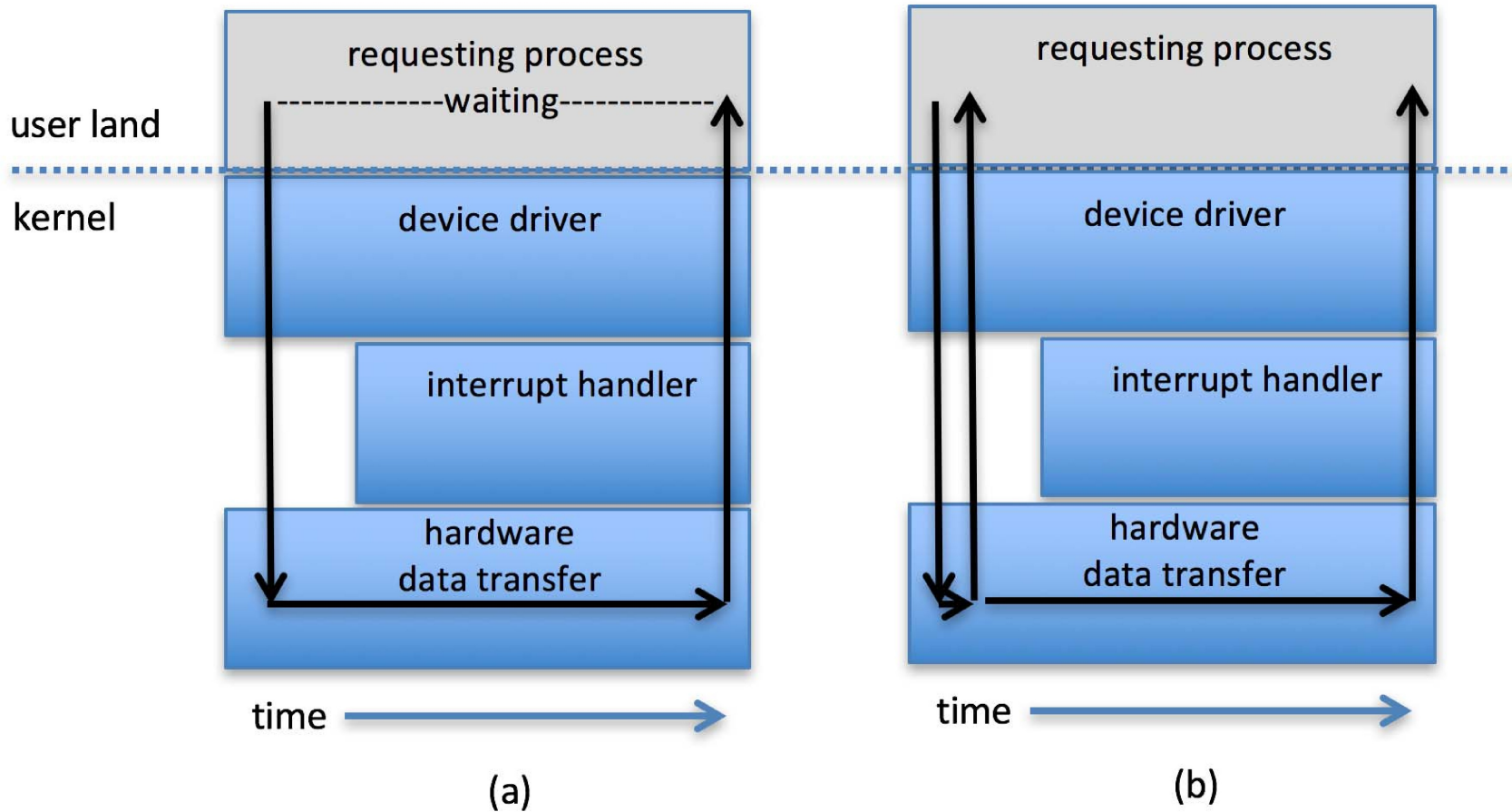


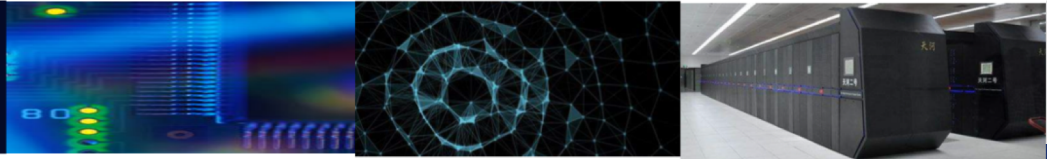
非阻塞和异步I/O

- ❖ **Blocking - process suspended until I/O completed**
 - Easy to use and understand
 - Insufficient for some needs
- ❖ **Nonblocking - I/O call returns as much as available**
 - User interface, data copy (buffered I/O)
 - Implemented via multi-threading
 - Returns quickly with count of bytes read or written
 - **select()** to find if data ready then **read()** or **write()** to transfer
- ❖ **Asynchronous - process runs while I/O executes**
 - Difficult to use
 - I/O subsystem signals process when I/O completed



两种I/O方式





Blocking/nonblocking; Synchronous/asynchronous; send/receive primitives

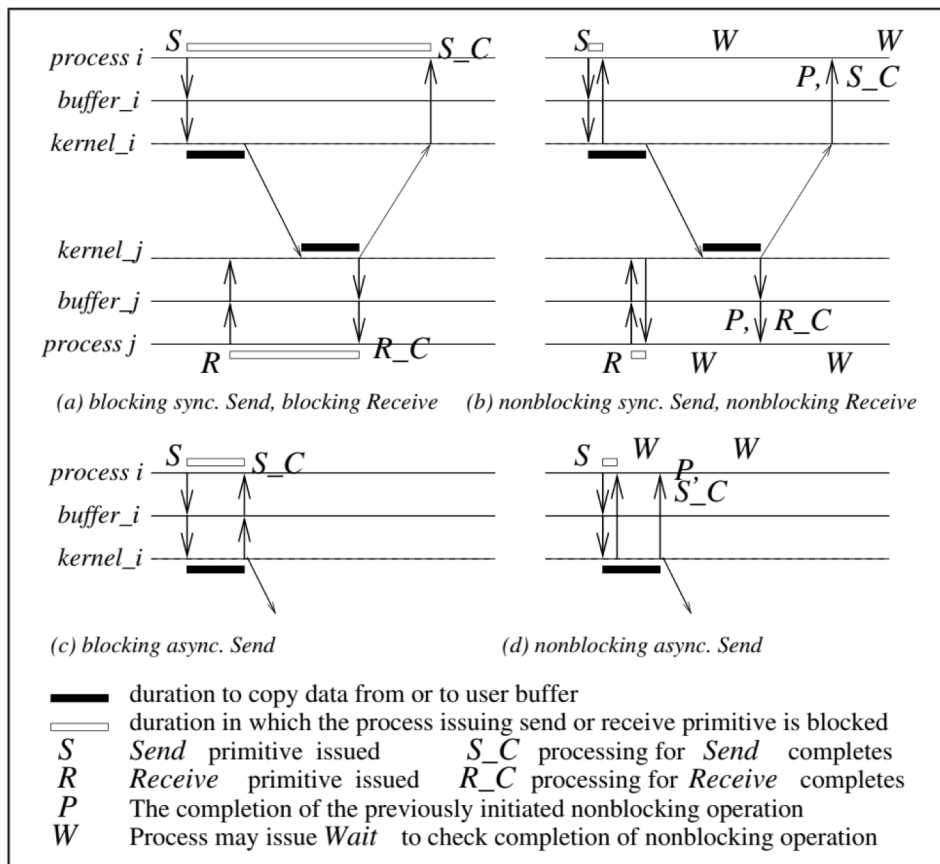


Figure 1.8: Illustration of 4 send and 2 receive primitives



向量 I/O

- ❖ **Vectored I/O allows one system call to perform multiple I/O operations**
- ❖ **For example, Unix `readve()` accepts a vector of multiple buffers to read into or write from**
- ❖ **This scatter-gather method better than multiple individual I/O calls**
 - Decreases context switching and system call overhead
 - Some versions provide atomicity
 - Avoid for example worry about multiple threads changing data as reads / writes occurring



内核I/O子系统

❖ Scheduling

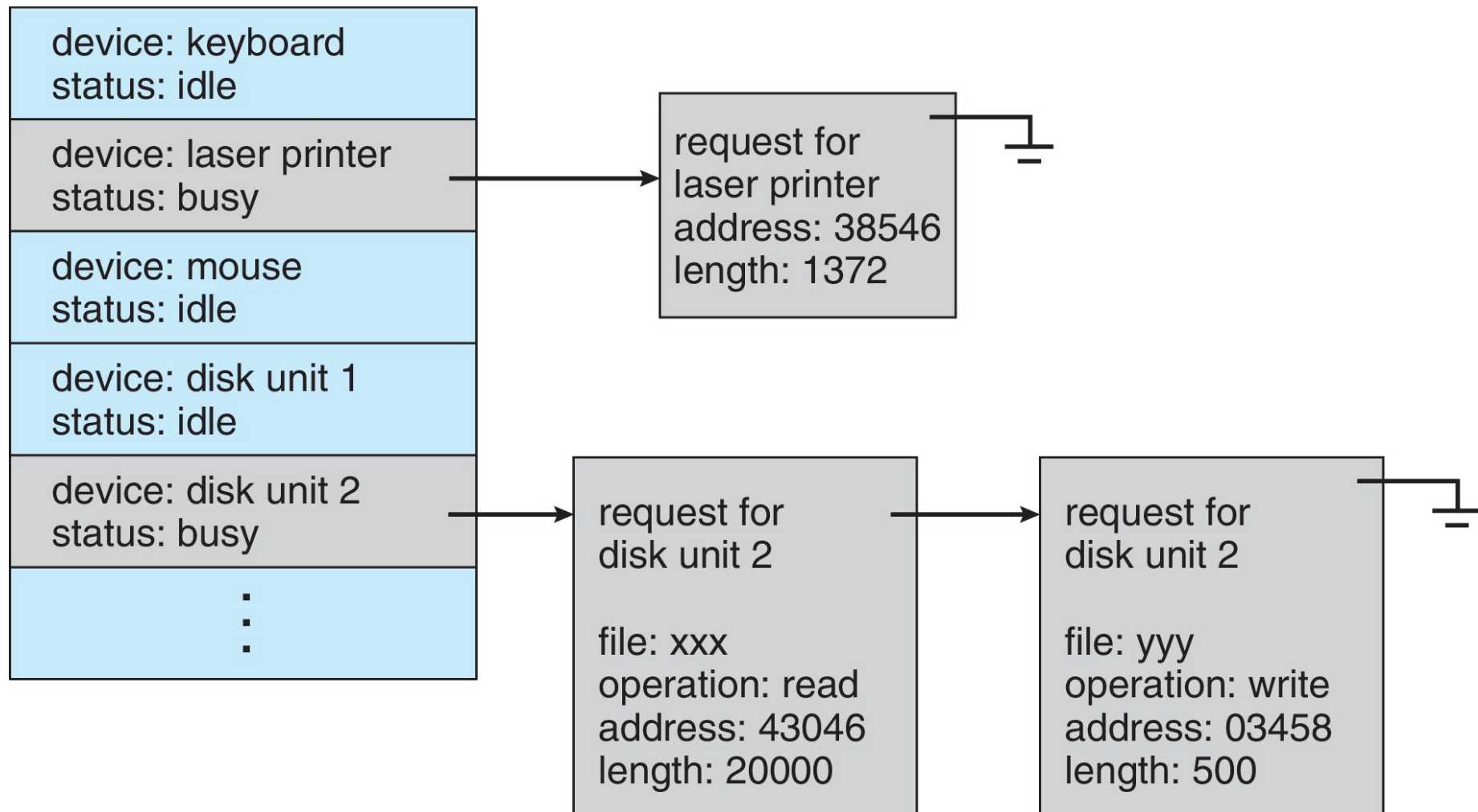
- Some I/O request ordering via per-device queue
- Some OSs try fairness
- Some implement Quality Of Service (i.e. IPQOS)

❖ Buffering - store data in memory while transferring between devices

- To cope with device speed mismatch
- To cope with device transfer size mismatch
- To maintain “copy semantics”
- **Double buffering** – two copies of the data
 - Kernel and user
 - Varying sizes
 - Full / being processed and not-full / being used
 - Copy-on-write can be used for efficiency in some cases

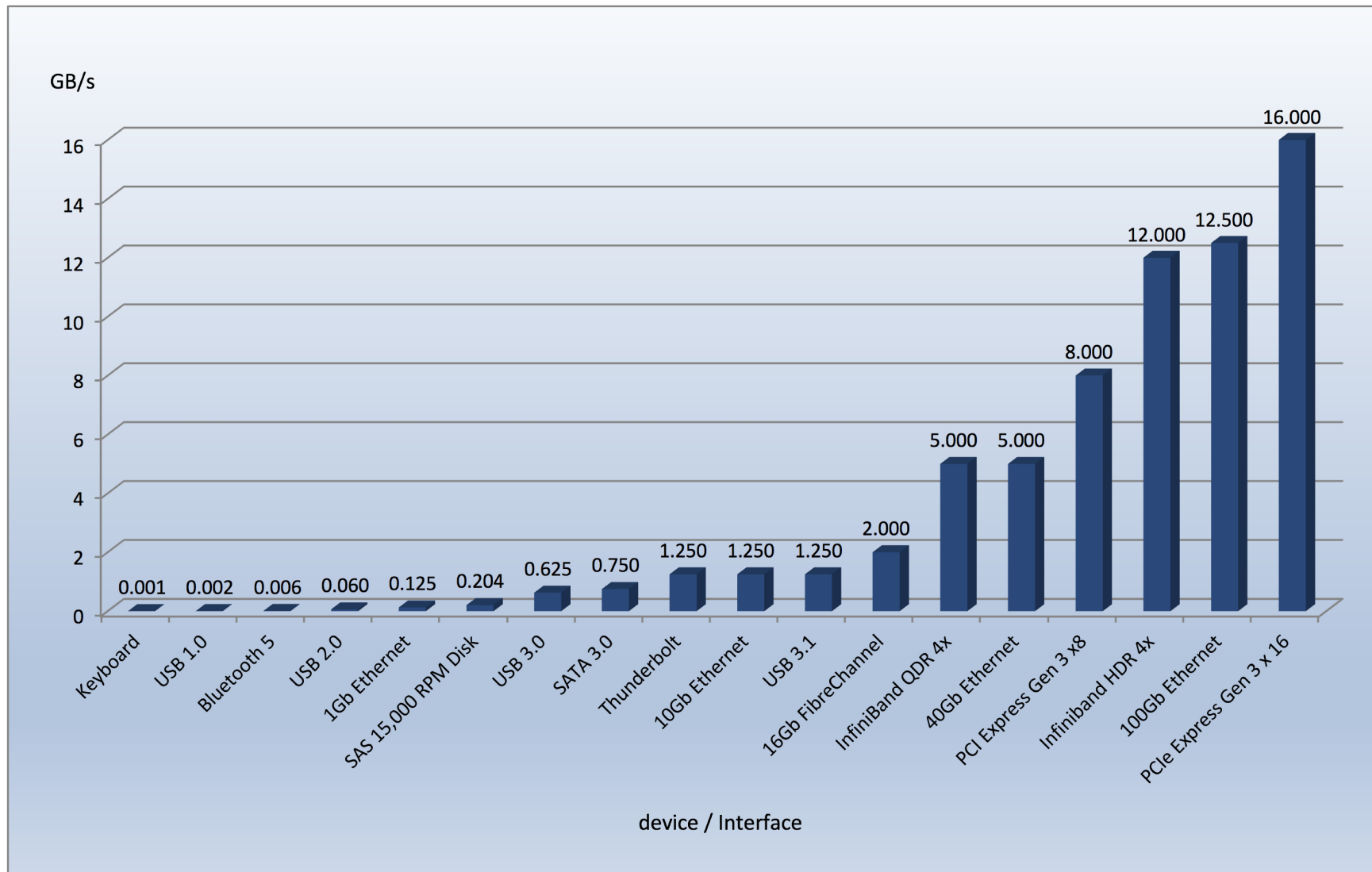


设备状态表





PC和数据中心I/O设备和接口速度





内核I/O子系统

- ❖ **Caching - faster device holding copy of data**
 - Always just a copy
 - Key to performance
 - Sometimes combined with buffering
- ❖ **Spooling(假脱机) - hold output for a device**
 - If device can serve only one request at a time
 - i.e., Printing
- ❖ **Device reservation (设备预留) - provides exclusive access to a device**
 - System calls for allocation and de-allocation
 - Watch out for deadlock



错误处理

- ❖ **OS can recover from disk read, device unavailable, transient write failures**
 - Retry a read or write, for example
 - Some systems more advanced – Solaris FMA, AIX
 - Track error frequencies, stop using device with increasing frequency of retry-able errors
- ❖ **Most return an error number or code when I/O request fails**
- ❖ **System error logs hold problem reports**



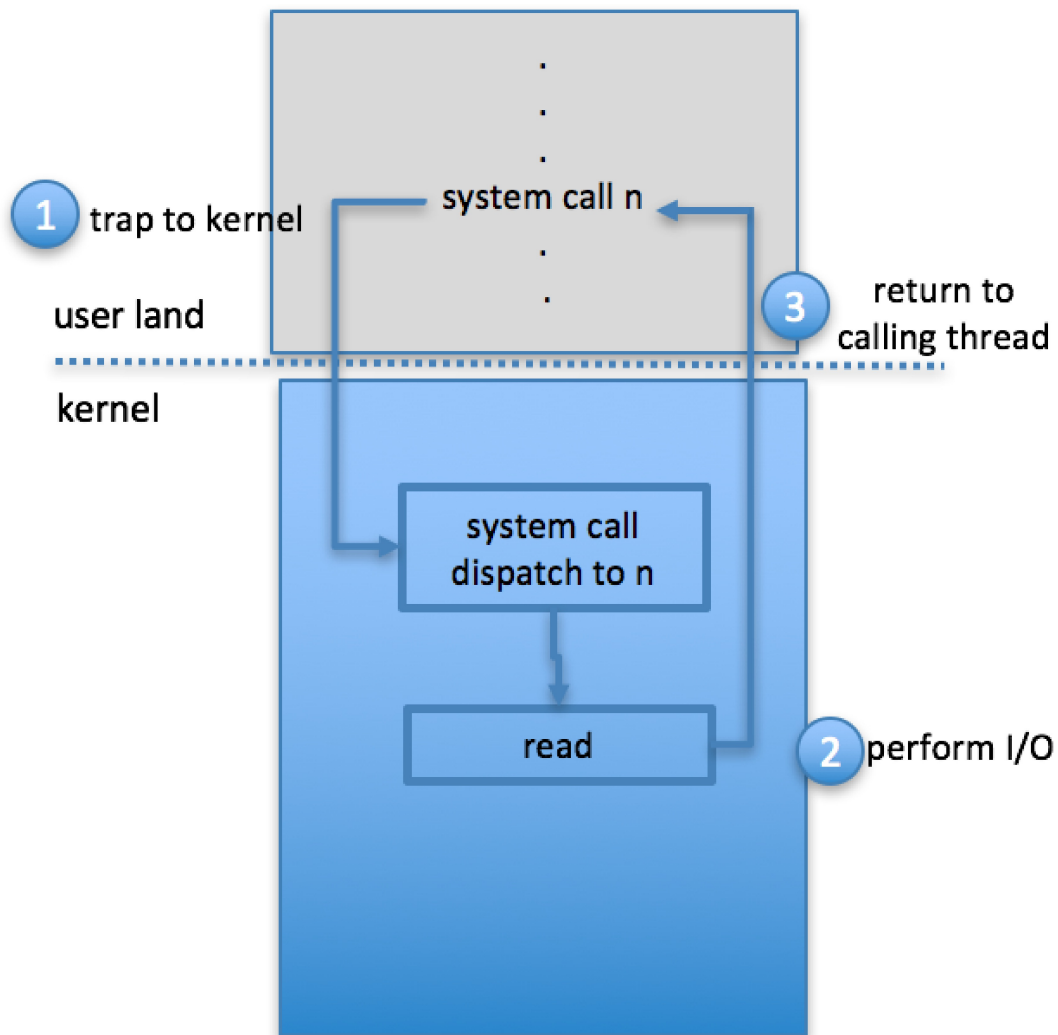
I/O保护

❖ User process may accidentally or purposefully attempt to disrupt normal operation via illegal I/O instructions

- All I/O instructions defined to **be privileged** (特权指令)
- I/O must be performed via **system calls**
 - Memory-mapped and I/O port memory locations must be protected too



利用系统调用执行I/O



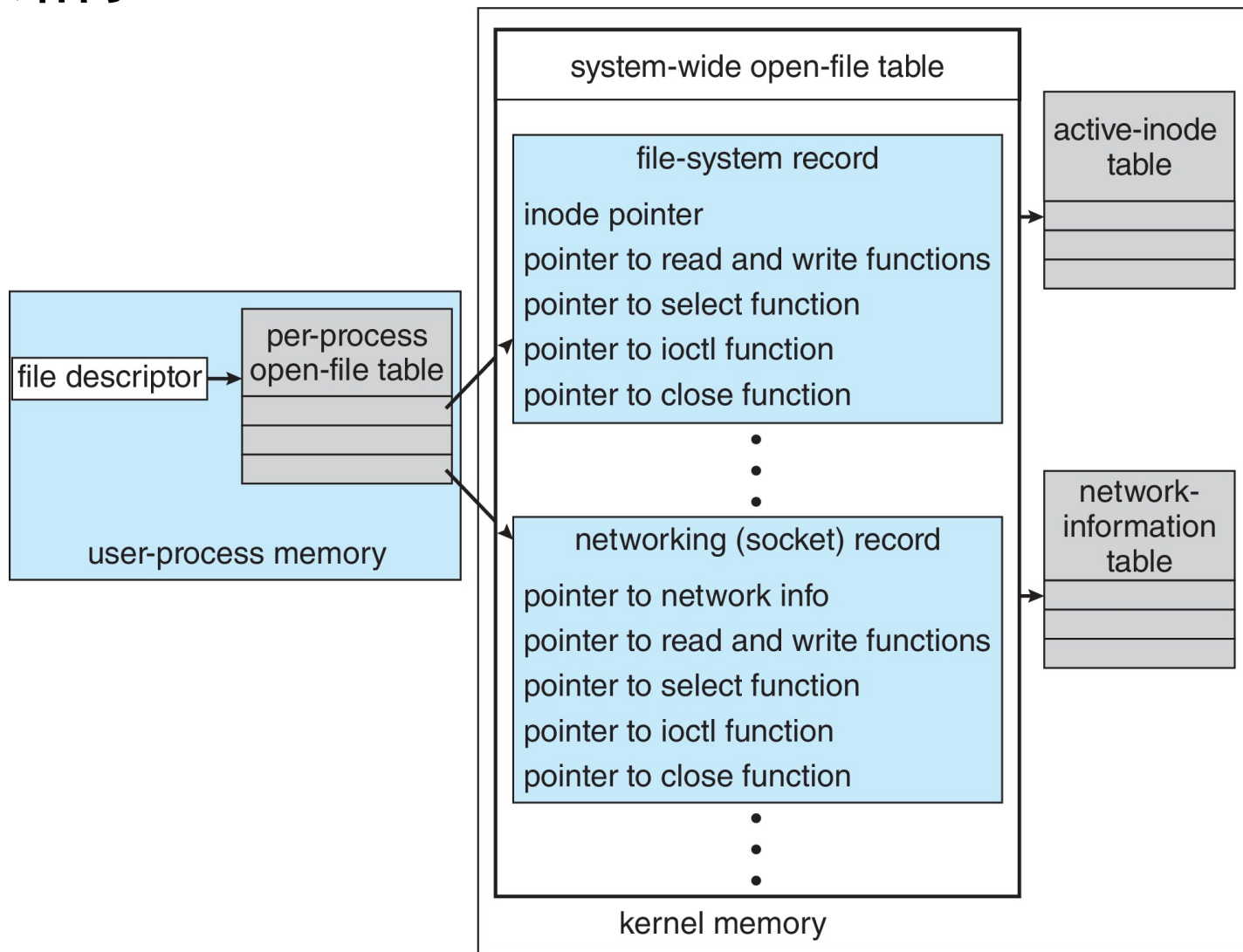


内核数据结构

- ❖ 内核保存I/O组件的状态信息，包括打开的文件表、网络连接、字符设备状态
- ❖ 很多复杂的数据结构来跟踪缓冲区、内存分配、“脏”块
- ❖ 有些使用面向对象的方法和消息传递来实现I/O
 - Windows**使用消息传递**
 - 带有从用户模式传递到内核的I/O信息的消息
 - 消息在流向设备驱动程序并返回到进程时被修改
 - 赞成/反对？与共享内存的数据结构相比有什么优势？



Unix I/O结构





电源管理

- ❖ 不是严格意义上的I/O领域，但很多是与I/O相关的
- ❖ 计算机和设备使用电，产生热量，经常需要冷却
- ❖ 操作系统可以帮助管理和改进使用
 - 云计算环境在服务器之间移动虚拟机
 - 最终可能会分散整个系统并关闭它们
- ❖ 移动计算将电源管理作为第一类操作系统功能



电源管理

❖ 例如，Android实现了

- 组件级电源管理

- 了解组件之间的关系
- 构建表示物理设备拓扑的设备树
- 系统总线→I/O子系统→闪存，USB存储}
- 设备驱动程序跟踪设备的状态，无论是否在使用中
- 未使用的组件 - 将其关闭
- 树分支中的所有设备未使用 - 关闭分支



电源管理

❖ 例如，Android实现（续）

- 唤醒锁 - 与其他锁一样，但在锁定时防止设备睡眠
- 电源崩溃 - 使设备进入深度睡眠
 - 边际用电
 - 只有足够清醒才能对外部刺激做出反应（按键、来电）

❖ 现代系统使用高级配置和电源接口（ACPI）固件，提供作为内核调用的例程运行的代码，用于设备发现、管理、错误和电源管理



I/O总结

- ❖ 总之，I/O子系统协调应用程序和内核其他部分可用的大量服务
 - 管理文件和设备的名称空间
 - 对文件和设备的访问控制
 - 操作控制（例如，调制解调器无法查找（）
 - 文件系统空间分配
 - 设备分配
 - 缓冲、缓存和假脱机
 - I/O调度
 - 设备状态监视、错误处理和故障恢复
 - 设备驱动程序配置和初始化
 - I/O设备的电源管理
- ❖ I/O子系统的上层通过设备驱动程序提供的统一接口访问设备

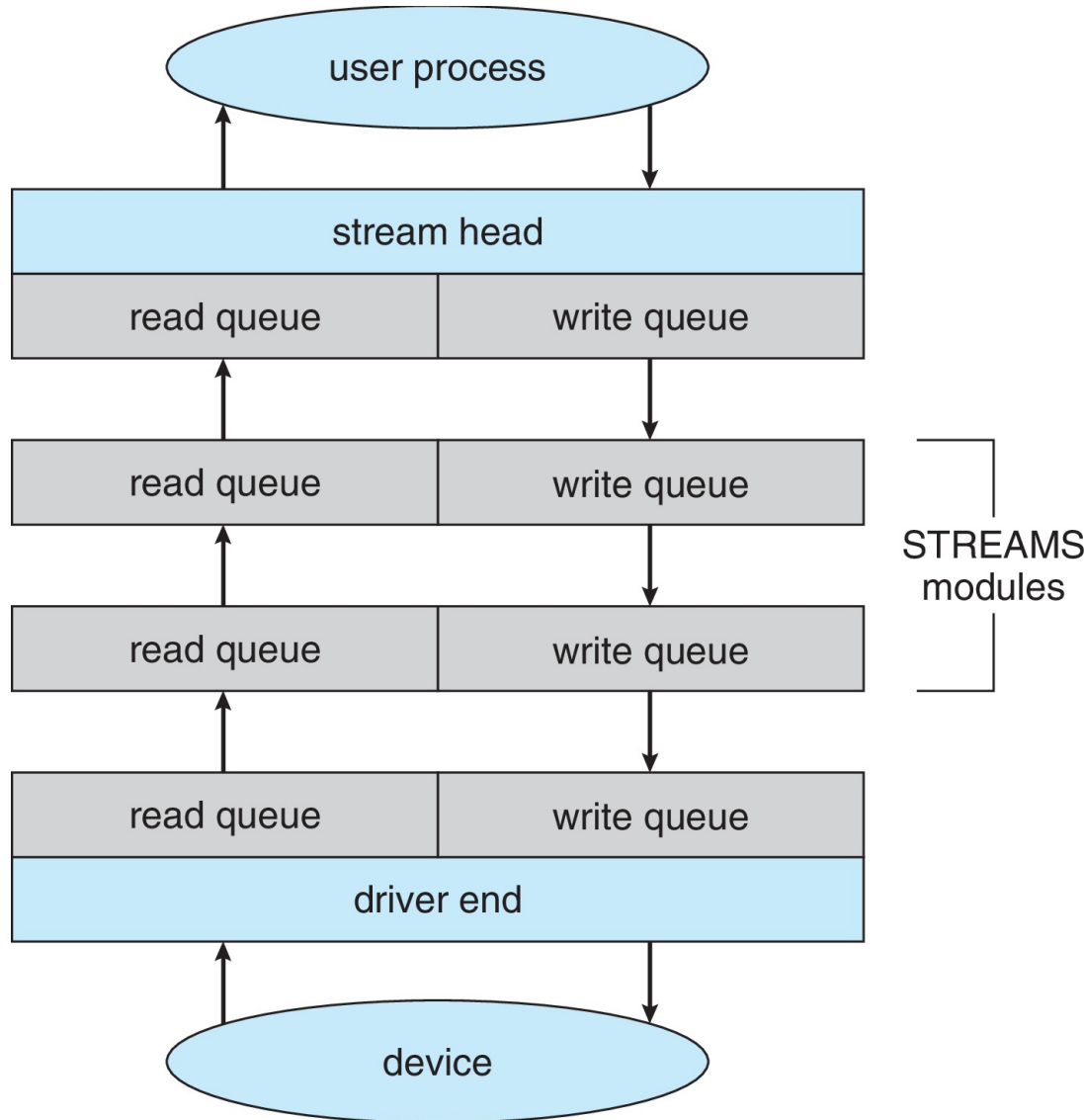


流式I/O

- ❖ STREAM - Unix System V及更高版本中用户级进程与设备之间的全双工通信通道
- ❖ 流包括:
 - 流头与用户进程接口
 - 驱动程序端与设备接口
 - 它们之间有零个或多个流模块
- ❖ 每个模块包含一个读队列和一个写队列
- ❖ 消息传递用于队列之间的通信
 - 流量控制选项, 指示可用或忙
- ❖ 内部异步, 用户进程与流头通信的同步



流式I/O



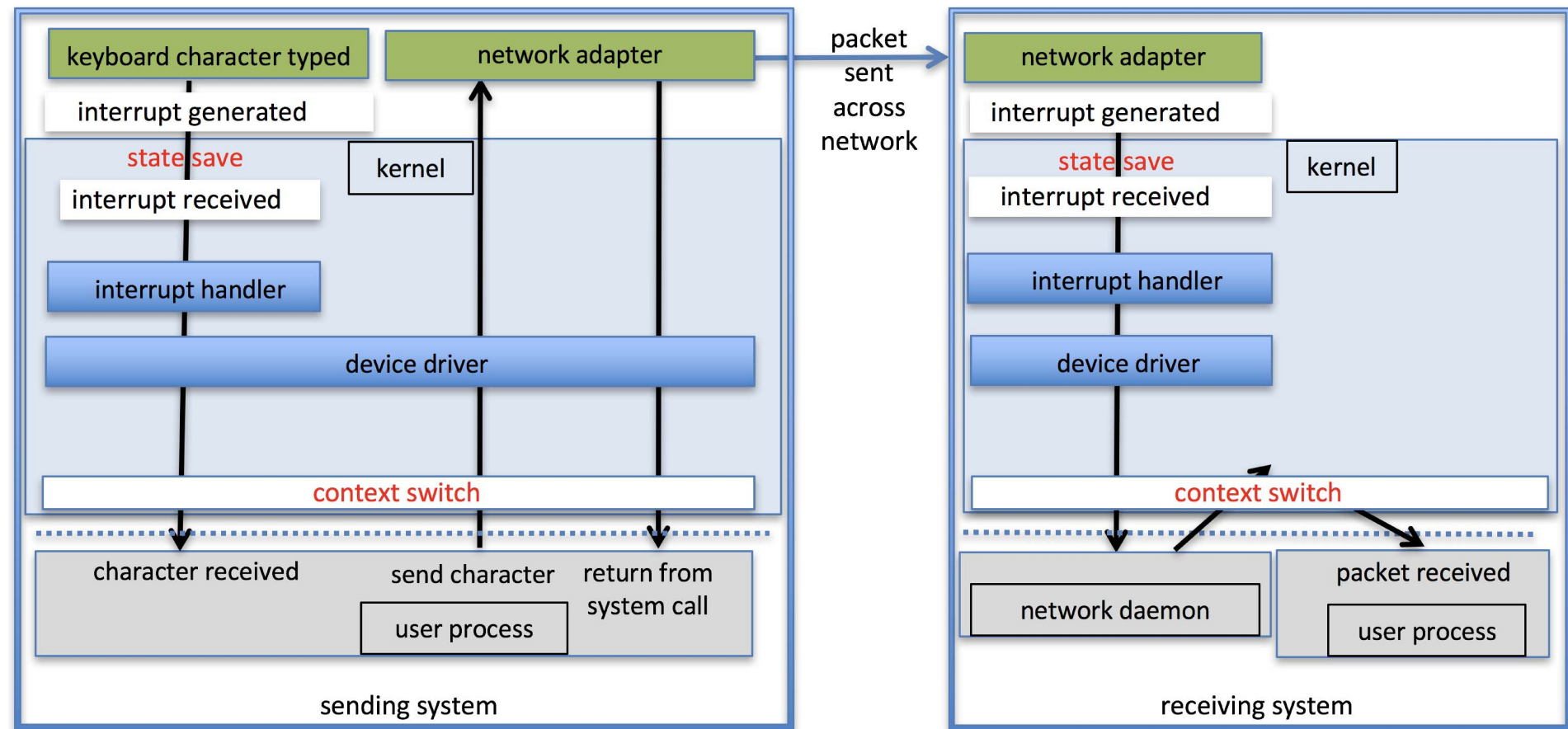


性能

- ❖ I/O是影响系统性能的主要因素：
 - 要求CPU执行设备驱动程序、内核I/O代码
 - 中断导致的上下文切换
 - 数据复制
 - 网络流量密度



内部计算机通信





性能改进

- ❖ 减少上下文切换的数量
- ❖ 减少数据复制
- ❖ 通过使用大型传输、智能控制器和轮询减少中断
- ❖ 使用DMA
- ❖ 使用更智能的硬件设备
- ❖ 平衡CPU、内存、总线和I/O性能以获得最高吞吐量
- ❖ 将用户模式进程/守护进程移动到内核线程



中山大學
SUN YAT-SEN UNIVERSITY

计算机学院 (软件学院)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



谢谢